

CAN Access

Programming  
Language





- 1 – *Introduction*
- 2 – *Event-driven software*
- 3 – *Browser CAPL*
- 4 – *Syntax*
- 5 – *How to use the messages*
- 6 – *Keyboard usage*
- 7 – *System event and timer*
- 8 – *Panel Designer*



# 1. Introduction



.CAPL

## Introduction

Based C programming language, CAPL (CAN Access Programming Language), is a language used only in CANalyzer and CANoe tools.

The scope behind the CAPL is to support the system based on CAN developer activity:

- Optimize the control of measurement and test task
- Simulation of all nodes of the system
- Support to analyze the CAN trace monitoring messages and events

Using CANalyzer or CANoe with CAPL you can create custom applications with user-defined behavior. The power of the application is limited only by imagination, limitations from the hardware available for communication and speed of the PC.



### Evaluation capability

- Analyze messages or specific data
- Analyze CAN trace
- Create or modify the environment
- Create a custom model for diagnostic tool
- Manage the replay block to analyze previous log
- Manage a gateway between two different CAN
- Evaluate the software strategy simulate the introducing of error frame

### Simulation capability

- Create a sort of black box to simulate system or nodes
- Simulate messages both cyclic and on event
- Simulate messages after user action (like press of a key for example)
- Timed nodes or network events
- Multiple temporal events, each with its own programmed behavior  
Cambiamenti dei parametri fisici o dei valori simbolici
- Network or module fault to evaluate strategies
- Create customized function

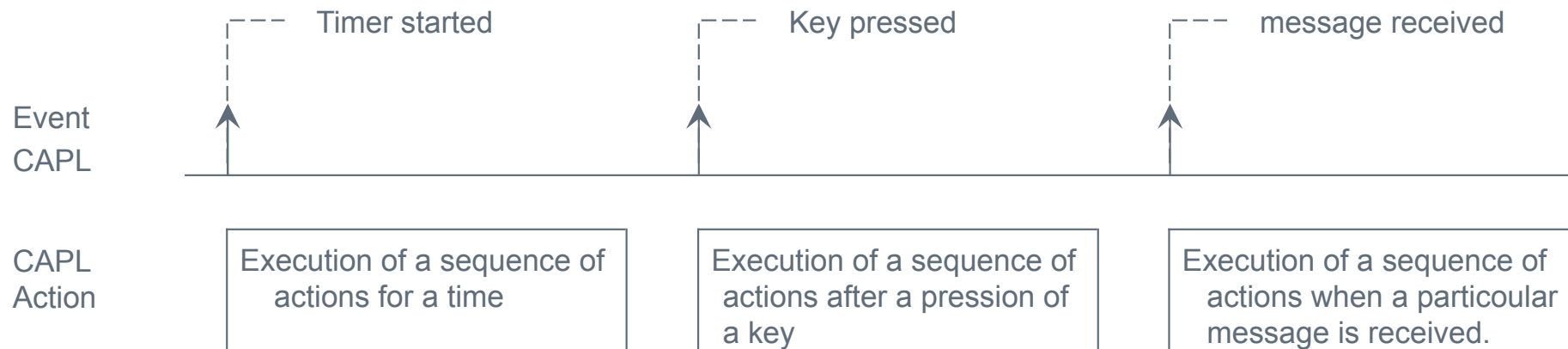


## 2. Event-driven software



## .CAPL Event-driven software

CAPL is an event-based language.



The CAPL script can be written to work forpuò essere creato in modo da rispondere a:

- On timer event (timer software)
- I/O event (keyboard, serial port...)
- CAN event (messages, errors...)



# 3. Browser CAPL

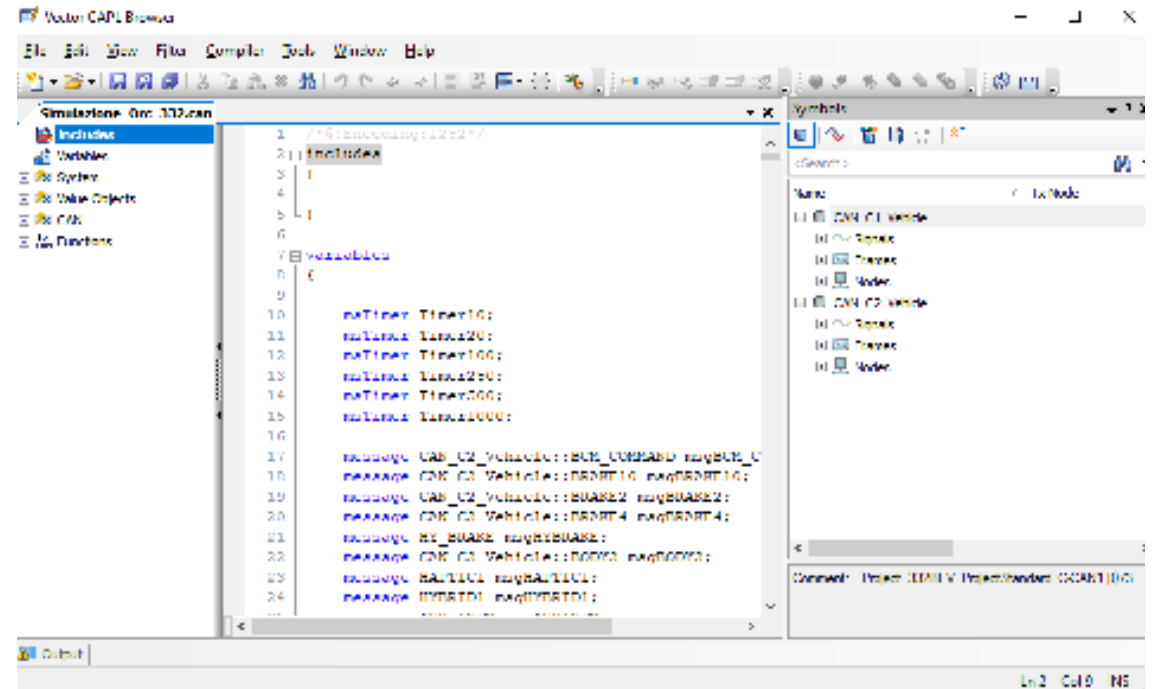


Browser -

CAPL

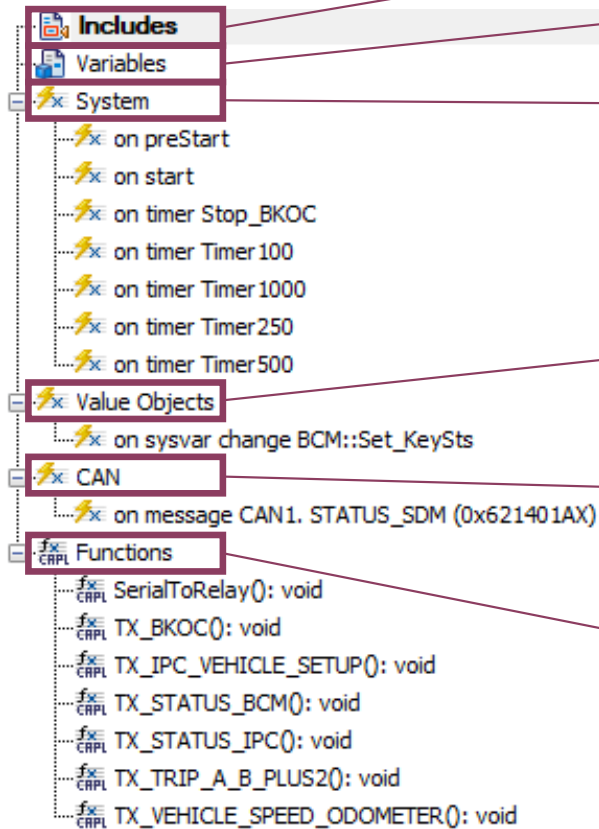
The CAPL Browser allows to organize, create, edit and modify the function. The Browser support the writing of the script and organize the script in a tree structure for each events.

The script can be compiled with this browser directly.





## Browser - CAPL



Allows to call external libraries

Define global variable

External Events definition

System variable Events definition

CAN event definition

Function definition



# 4. Sintax



## Sintax

## - CAPL

Keyword “this”-> is the reference to an object that is the trigger for the event:

- on message
- on envVar
- on key
- on errorframe
- on BusOff...

The Keyword “this” is a sort of pointer to an object

Example:

**on message 555**

```
{
    byte val;
    val = 0;
    if (this.CAN == 1) // if the message is received on CAN 1
    {
        val = this.byte(0); // “val” is equal to the first byte of CAN message payload
    }
}
```



# Syntax

## - CAPL

| Symbol | Operation        | Example (y =8; z=4)  |
|--------|------------------|----------------------|
| +      | Sum              | x = y + z; // x = 12 |
| -      | Subtraction      | x = y - z; // x = 4  |
| -      | Negative         | y = - y // y = -8    |
| *      | factor           | x = y * z; // x = 32 |
| /      | Division         | x = y / z; // x = 2  |
| %      | Rest of division | x = y % z; // x = 0  |
| ++     | Post-Increment   | x = y++; // x = 8    |
| ++     | Pre-Increment    | x = ++y; // x = 9    |
| --     | Post-Decrement   | x = z--; // x = 4    |
| --     | Pre-Decrement    | x = --z; // x = 3    |

| Symbol | Operation | Example<br>(y =0x00; z=0x01) |
|--------|-----------|------------------------------|
| !      | NOT       | if(!y) // TRUE               |
|        | OR        | if(y==0 && z==1) // TRUE     |
| &&     | AND       | if(y==1    z==1) // TRUE     |

| Symbol | Operation      | Example (y =8; z=4) |
|--------|----------------|---------------------|
| =      | Assign         | y = z // y = 4      |
| +=     | Sum and assign | y +=z; // y = 12    |
| &=     | AND            | y &= z; // y = 0    |
| =      | OR             | y  = z; // y = 12   |
| ^=     | XOR            | y ^= z; // y = 12   |



## Sintax

## - CAPL

| Data Type | Description                       | Size   | Unsigned/Signed |
|-----------|-----------------------------------|--------|-----------------|
| char      | Character                         | 8 bit  | unsigned        |
| byte      | Byte                              | 8 bit  | unsigned        |
| int       | Integer                           | 16 bit | signed          |
| word      | Word                              | 16 bit | unsigned        |
| long      | Long integer                      | 32 bit | signed          |
| dword     | Double word                       | 32 bit | unsigned        |
| float     | Single precision floating point   | 64 bit | signed          |
| double    | Single precision floating point   | 64 bit | signed          |
| message   | Communication message             | --     | --              |
| timer     | timer with second resolution      | --     | --              |
| mstimer   | timer with millisecond resolution | --     | --              |



# Sintax

## - CAPL

| Istruction | Format                                                                                                                                         |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| If         | if (espressione) istruzione; if (espressione)<br><br>{<br>istruzione;<br>}                                                                     |
| If-Else    | if (espressione) istruzione1; else istruzione2;                                                                                                |
| Switch     | switch (espressione) {<br>case value1: istruzione1; break; case<br>value2: istruzione2; break;<br><br>...<br>default: istruzioneX; break;<br>} |

| Istruction | Format                                                                                                                                         |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| If         | if (espressione) istruzione; if (espressione)<br><br>{<br>istruzione;<br>}                                                                     |
| If-Else    | if (espressione) istruzione1; else istruzione2;                                                                                                |
| Switch     | switch (espressione) {<br>case value1: istruzione1; break; case<br>value2: istruzione2; break;<br><br>...<br>default: istruzioneX; break;<br>} |



# 6. How to use the Messages



## How to use the Messages

To manage the messages with CAPL it is important know if the message is transmitted or received.

If a DBC is loaded in the CANalyzer or CANoe configuration it is possible call the desired message. The l'ID, DLC, signals and other properties are imported automatically.

**Sintax:**     **message <id or message name> <variable name>**

```
message STATUS_BCM msgSTS_BCM;
```

```
// it is possible use even the ID
```

```
message 0x100 msgSTS_BCM;
```





## How to use the Messages

It is possible define one message that does not exist in the DBC with an ID address.

```
message 0x100 msgSTS_BCM;
```

Every property should be defined, in the following example DLC is defined:

```
message 0x100 msgSTS_BCM = {DLC = 8}; //DLC assigned during the declaration  
msgSTS_BCM.DLC = 8; //DLC assigned after
```



## How to use the Messages

Every message have signals. The signal have hexadecimal value on CAN.

To have the physical value of the signal (applying the offset or factor if are defined in the DBC) it is possible use the keyword '**phys**'

*Physical Value = (Raw Value \* Factor) + Offset*

**message EngData EDMsg;**

**EDMsg.EngSpeed = 5000;**

// the value is without conversion (if Factor = 2 e Offset = 0, on the bus we have 2500 in hexadecimal 0x9C4)

**EDMsg.EngSpeed.phys = 5000;**

// physical value



## How to use the Messages

When we want change dinamically in the real time the value of a certain signal it is possible associate the signal to a **system variable** externally defined (on CANalyzer or on CANoe).

This way to proceed is useful when we want use a Panel interface.

To associate the system variable the syntax is the following **@sysvar::<Name Space>::<Name variable>;**

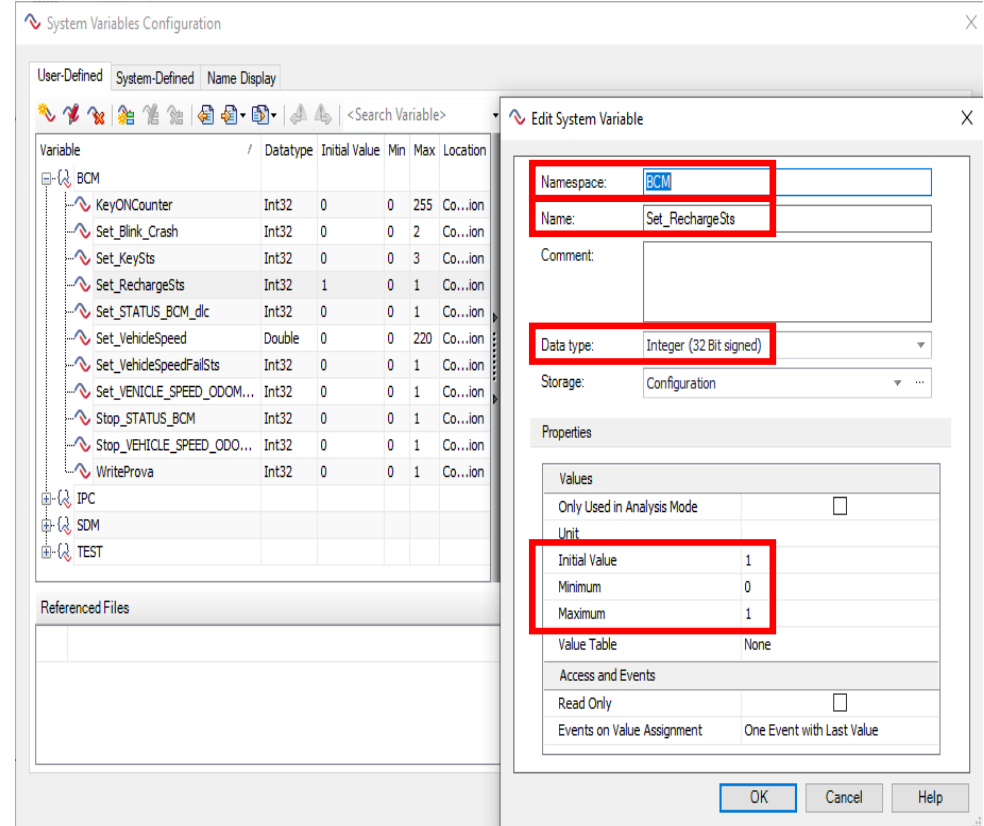
```
msgSTATUS_BCM.RechargeSts = @sysvar::BCM::Set_RechargeSts;
```

Per poter definire la variabile di sistema Set\_RechargeSts, è necessario aprire CANalyzer sulla scheda «Enviroment», quindi scegliere «System Variables» ed aprire il tab «User-defined». Cliccando su «New System Variable», va definito un namespace (ad esempio il nome della centralina interessata), il nome della variabile, il tipo di variabile, il valore di default e i valori minimo e massimo.



## How to use the Messages

To define the system variable 'Set\_RechargeSts', It is possible open the layer «Environment» on CANalyzer, select «System Variables» open the tab «User-defined». Click on «New System Variable», define a namespace (for example «BCM»), the variable name, the variable type, the default value and maximum and minimum value.





## How to use the Messages

To transmit the message CAPL implement a specific function called «output»

In the following it is created a function «TX\_STATUS\_BCM» in which are manipulated 2 signals and then the message is transmitted with the «output»:

```
message    STATUS_BCM msgSTS_BCM
```

```
TX_STATUS_BCM
```

```
{
```

```
msgSTS_BCM.VehicleSpeed.phys = @sysvar::BCM::Set_VehicleSpeed;
```

```
msgSTATUS_BCM.RechargeSts = 1;
```

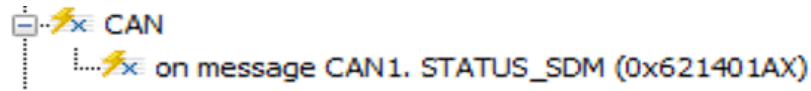
```
output(msgSTS_BCM);
```

```
}
```



## How to use the Messages

One of the event definition more useful in CAPL is the «on message». The section in the tree structure in which we can find it is «CAN»



```
on message STATUS_SDM //the following instruction are executed after that «STATUS_SDM» is received
{
  If (this.AirBagFailSts == 2) //with the pointer this. It is controlled «AirBagFailSts» is equal to 2

  {
    @sysvar::IPC::Set_warningLamp = 1; // variable Set_warningLamp is set
  }
}

on message CAN2.* // in this case is monitored if on CAN 2 is received any message not one in particular
{

  if(this.dir != rx) return;
}
```



## How to use the Messages

There are even other event based on can that can be managed with CAPL for example «On Error frame» «on Bus Off»...

In the error frame definition there aren't info about the cause of the error or the transmitter. It is used to statistic evaluation of the CAN

**on errorFrame** //if errorframe is received execute the following instruction

```
{  
if (this.can == 1 && (timeNow() – pretime) < 100000)  
write(“Error frames received less than one second apart on channel 1.”);  
}
```

It is also possible generate Error Frame

```
output(errorframe);
```



# 7. Keyboard usage



## Keyboard usage

The instructions are executed when a key of the keyboard is pressed.

| Tasto              | Procedura           |
|--------------------|---------------------|
| a                  | on key 'a'          |
| A                  | on key 'A'          |
| A                  | on key 0x41         |
| 2                  | on key '2'          |
| \$                 | on key '\$'         |
| End                | on key End          |
| Shift + F1         | on key shiftF1      |
| Control + PageDown | on key ctrlPageDown |
| <i>Qualsiasi</i>   | on key *            |

```
on key 'a' // pressing the key a
{
    msgSTATUS_BCM.RechargeSts = 1;
}
```



## 8. System event and timer



## System event and timer

### SYSTEM EVENT

CAPL have the possibility to trigger the execution using some system event :

- Before of the starting – on prestart □ typically used to read file, initialize variable print something in the write window...
- At starting (when Start Button is pressed) - on start □ Typically used to initialize the timer send message on the bus...
- At Stop (when Stop Button is pressed) – on stopMeasurement □ Typically used for manage the log write information ...



## System event and timer

### TIMER

The timer give an easy way to trigger periodic message. They are a sort of programmable clocks  
To use the timer there are three step to perform:

- Timer declaration

#### variables

```
{      msTimer myTimer; // crea un timer in millisecondi  
      message 100 msg; // crea il messaggio 100
```

- Set the timer

#### on key 'a'

```
{  
  setTimer(myTimer,20); // imposta il timer }
```

- Define the instruction to execute at on timer event

#### on timer myTimer // when on timer is set

```
{      output(msg); // send the message
```



# 9. Panel Designer

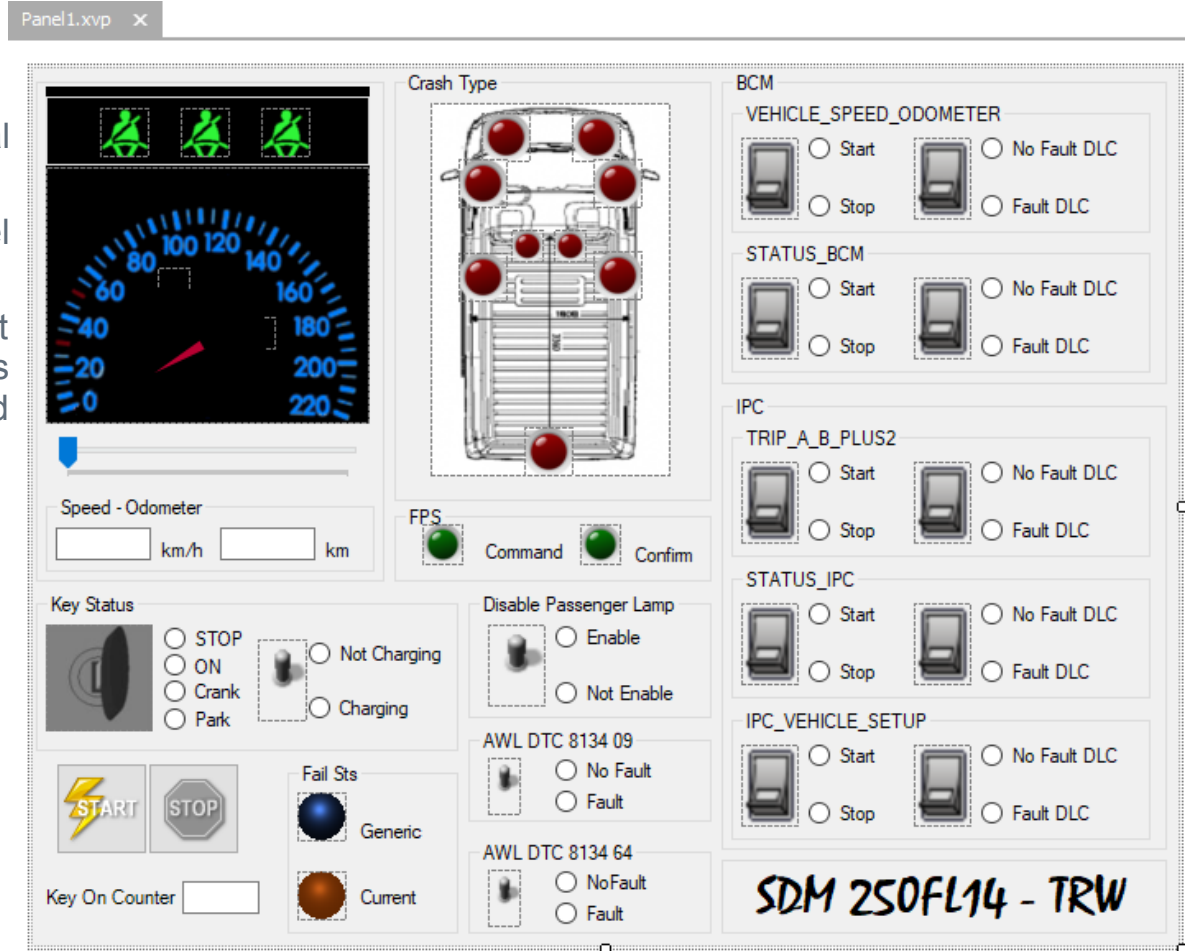


## Panel Designer

CANalyzer give the possibility to create a graphical to manage the simulation.

If you go on «Tools» in CANalyzer, select «Panel Designer».

In this way you have an editon on which insert switch led icon and so on. To this variable it is possible associate the system variables created and use it to monitor and manipulate the can signal.

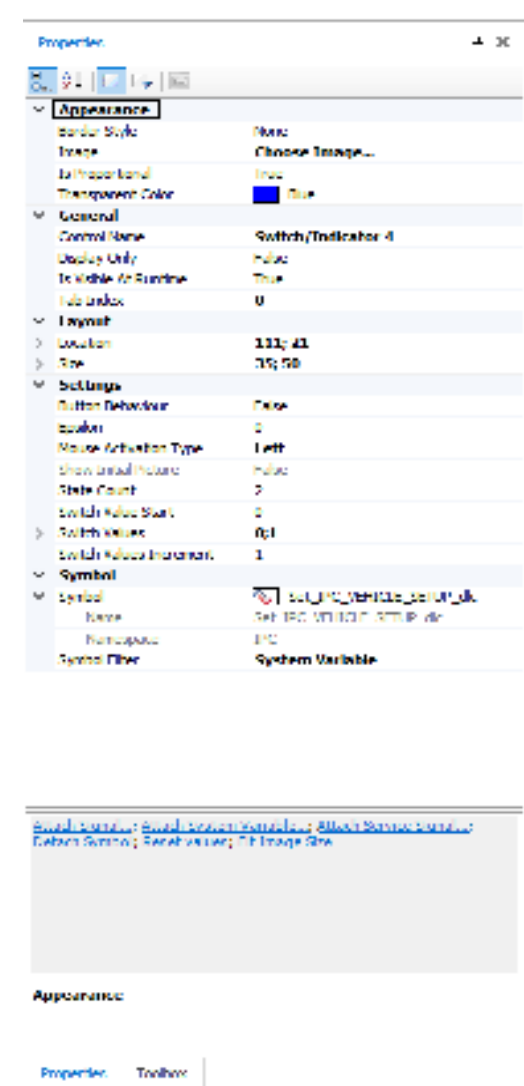
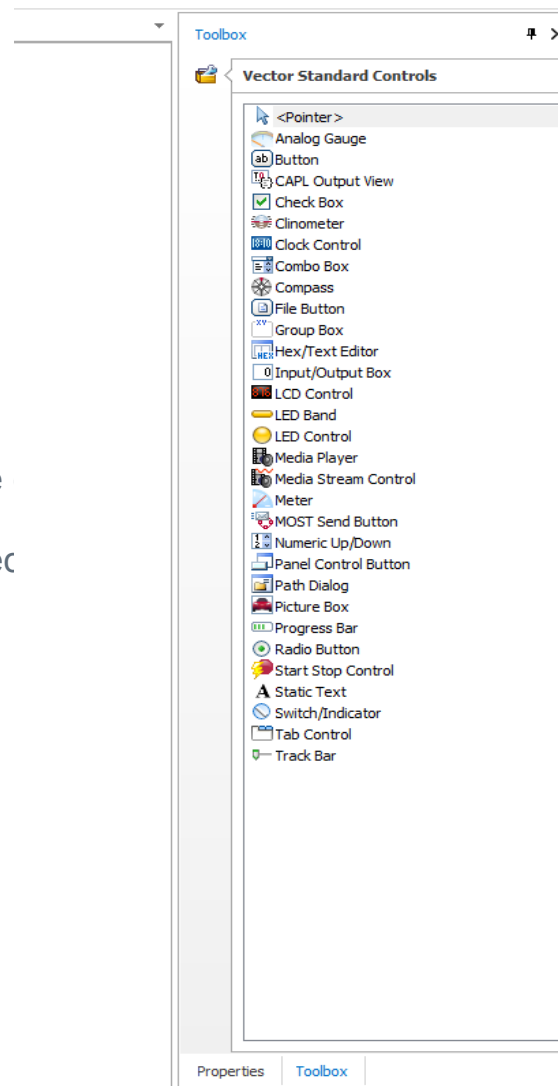




## Panel Designer

In the tab «Toolbox» it is possible choose the that I need;

We can associate to the icon a system variable signal (section «Symbol» in the «properties» and it is possible change the graphic in the sec «Appearance».



# CONTACTS

Martina Luccaro

[martina.luccaro@kineton.it](mailto:martina.luccaro@kineton.it)

Via G. Porzio,  
Centro Direzionale Isola B3  
80143 Napoli Via G. Porzio

[www.kineton.it](http://www.kineton.it)

